

AI Chessathon Agent: High-Level Briefing

High-level explanation of the current submitted chess agent architecture.

One-Sentence Summary

Our current agent is a classical chess engine written in Python. It receives a board position and clock value, searches possible move sequences using a tree-search algorithm, scores resulting positions with a hand-written evaluation function, and returns a legal UCI move.

What We Submitted

- Main file: ``agent.py``
- Public API: ``get_move(fen: str, time_left_ms: int) -> str``
- Output format: UCI chess move, for example ``e2e4`` or ``e7e8q``
- Dependencies used by the agent: Python standard library and ``python-chess``
- Approach: classical search, not a neural model
- Runtime assumption: CPU-only competition environment

The implementation is designed to be explainable, legal-move safe, and easy to modify for experiments.

Core Idea

The agent does not memorize moves and does not ask an external system what to play. Instead, it performs local search from the current board state.

At a high level:

- 1 Parse the FEN string into a chess board.
- 2 Generate all legal moves.
- 3 Pick a legal fallback move before doing expensive computation.
- 4 Allocate a conservative amount of time for search.
- 5 Search candidate move trees with iterative deepening.
- 6 Use alpha-beta pruning to avoid searching branches that cannot affect the decision.
- 7 Use quiescence search to reduce tactical horizon errors.
- 8 Evaluate leaf positions with chess heuristics.
- 9 Return the best legal move found before the deadline.

Architecture Flow

```
Competition platform
-> get_move(fen, time_left_ms)
-> parse FEN with python-chess
-> generate legal moves
-> choose fallback move
-> allocate time budget
-> iterative deepening search
    -> negamax alpha-beta
    -> transposition table lookup/store
    -> move ordering
    -> quiescence search
    -> static evaluation
-> verify move is legal
-> return UCI move
```

Search Method

The main search method is negamax with alpha-beta pruning.

Negamax is a compact form of minimax for zero-sum games. Since chess is zero-sum, a good position for the side to move is equally bad for the opponent. This lets the agent use one scoring function and flip the sign when turns alternate.

Alpha-beta pruning improves efficiency. If the search proves that a branch cannot change the final decision, that branch is skipped. This lets the agent search deeper in the same clock budget.

Iterative Deepening

The agent searches depth 1, then depth 2, then depth 3, continuing until the time budget is nearly exhausted.

This is useful because the agent always has a completed answer available. If depth 5 starts but times out, the agent can still return the best move from depth 4.

Evaluation Function

When the search reaches a leaf position, the agent estimates how good that position is.

Current evaluation terms:

- Material balance: pawns, knights, bishops, rooks, and queens.
- Piece-square tables: rewards pieces for occupying useful squares.
- Activity: rewards pieces that attack more squares.
- Pawn structure: penalizes doubled and isolated pawns.
- Passed pawns: rewards pawns that have a clearer path to promotion.
- Bishop pair: rewards owning two bishops.
- Terminal states: checkmate, stalemate, insufficient material, and fifty-move positions.

The score is approximately centipawn-like, but it is not calibrated to a professional engine.

Quiescence Search

Basic fixed-depth search can stop in the middle of a capture sequence and badly misread the position. Quiescence search reduces this problem.

At the normal depth limit, the agent continues searching tactical continuations such as captures and promotions. If the king is in check, it searches legal evasions. This gives the final evaluation a more stable tactical position.

Time Management

The agent uses a soft deadline inside ``get_move``.

Important behavior:

- If the clock is extremely low, return the fallback move immediately.
- Otherwise, allocate only part of the remaining clock.
- Check the deadline frequently during search.
- Stop search cleanly if the deadline is reached.

This is designed to reduce losses from flagging or timeout while still using available thinking time.

Safety And Robustness

The implementation is defensive in several ways:

- It selects a legal fallback move before search begins.
- It verifies the final move is legal before returning.
- It catches unexpected exceptions and falls back to a legal move.
- It pairs every ``board.push()`` with ``board.pop()`` using ``try/finally``.
- It bounds quiescence depth.
- It bounds the transposition table size.

The practical goal is simple: for any valid non-terminal FEN, return a legal move quickly enough.

What The Agent Does Not Do

The current version does not use:

- Reinforcement learning.
- A neural network.
- GPU inference.
- Opening books.
- Endgame tablebases.
- External chess engines.

- Network calls.
- Learned policy priors.

This is deliberate. The current version is the clean classical baseline for future experiments.

How To Explain It In A Meeting

Short explanation:

The agent is a conventional search-based chess engine. It expands legal moves from the current board, searches likely continuations with alpha-beta pruning, evaluates resulting positions using hand-written chess heuristics, and returns the best legal move it found within a conservative time budget.

More technical explanation:

The public entrypoint is `get_move(fen, time_left_ms)`. Internally it uses `python-chess` for board legality and move generation. The engine runs iterative deepening over a negamax alpha-beta search, stabilized by capture-focused quiescence search. Leaf positions are scored by material, piece-square tables, activity, pawn structure, passed pawns, bishop pair, and terminal-state rules. Move ordering and a transposition table make the search more efficient. A fallback move is chosen before search so the agent can still respond legally under timeout or unexpected failure.

Current Status

This is submission-ready as a baseline agent:

- It is integrated into the starter repo's real `agent.py`.
- It preserves the required competition API.
- It packages as a zip with `agent.py` at the archive root.
- It is understandable enough to explain to judges or teammates.

It is not claimed to be a final high-strength engine. It is the current baseline from which tuning, testing, and stronger search/evaluation experiments should proceed.

Generated from docs/CURRENT_AGENT_HIGH_LEVEL.md.